

Python Interview Theory Cheat Sheet

Definitions, examples, and reasoning, built from Automate the Boring Stuff practice code

How to use this sheet

Each topic below follows the same four-part structure: what it is, what it looks like in code, why Python is designed this way, and where it helps or hurts you. This is theory first, code second. The goal is being able to explain these concepts out loud in an interview, not just recognize the syntax.

1. Functions

Definition

A function is a named, reusable block of code that runs only when called. It can accept input (parameters), do work, and hand back output (a return value). Functions are Python's basic tool for breaking a program into smaller, testable pieces instead of writing one long script top to bottom.

Example

```
def add_name(name):  
    print('bethel' + name)  
  
add_name('ben stokes')  
add_name('Morgan')
```

Here `add_name` is defined once and called twice with different input. The alternative, copying the `print` line by hand for every name, is exactly what functions exist to avoid.

Why Python does it this way

Python treats functions as first-class objects. You can pass them around, store them in variables, and call them later, because under the hood a function is just another object with a `__call__` behavior. This is why `def` creates a name in the current scope rather than a special kind of keyword-only construct. A function is data, not magic.

Advantages

- Code you write once can run many times with different inputs, so you fix bugs in one place instead of five.
- A function name documents intent. `random_number_game(4)` tells you more than four lines of inline logic would.
- Functions are the unit interviewers actually test. Isolate the logic and you can unit test it without running the whole program.

Disadvantages

- Too many tiny functions can make a beginner's code harder to trace, since you have to jump around to follow execution.
- Functions that rely on global state (see Section 3) look reusable but secretly aren't. Calling them from a different context produces different results.
- Every function call adds a small amount of overhead in exchange for readability, though for almost all real programs, this is irrelevant.

2. Arguments vs. Parameters

Definition

A **parameter** is the variable name written inside the function definition. An **argument** is the actual value supplied when the function is called. People use the words interchangeably in casual conversation, but interviewers like to hear the distinction stated correctly.

Example

```
def random_number_game(number):    # 'number' is the parameter
    if number == 1:
        return 'Australia ranked number 1 team in test'
    # ...

random_number_game(3)              # '3' is the argument
```

Why the distinction matters

The parameter is a placeholder that exists only while the function's code is being defined and read. It has no value of its own. The argument is what fills that placeholder at the moment of the call. Confusing the two usually shows up as a bug: assuming a parameter has a value before any argument has been passed to it.

Advantages of parameterizing code

- One function, many outcomes: `random_number_game` produces four different strings depending on what argument it receives, without four separate functions.
- Testing gets easier: you can call the same function with edge-case arguments (0, negative numbers, `None`) to see how it behaves.

Disadvantages / pitfalls

- If a function doesn't handle every possible argument (see the `hellofunc.py` example, which has no `else` branch), it silently returns `None` instead of raising an error, a quiet bug rather than a loud one.
 - Parameters with no default value force the caller to always supply an argument, which can make simple calls verbose if most calls use the same value anyway.
-

3. Return Values and None

Definition

`return` sends a value back out of a function to whatever called it, and immediately ends the function's execution. If a function reaches its end without hitting a `return` statement, or if the `return` statement itself has no value after it, Python automatically hands back the special value `None`.

Example

```
def random_number_game(number):
    if number == 1:
        return 'Australia ranked number 1 team in test'
    elif number == 2:
        return 'England is not qualify for WTC final'
    # no else clause: if number is 5, nothing matches

result = random_number_game(5)
print(result)    # None
```

Why None exists

Python needs a way to represent “no meaningful value” that can never be confused with a real value like 0, '', or False. `None` is its own type (`NoneType`) with exactly one instance, so `x is None` is always a safe, unambiguous check, unlike checking `x == 0`, which would also be true for the number zero.

Advantages

- A function returning `None` on failure lets calling code check `if result is None:` and handle the failure explicitly, rather than crashing.
- Because `None` is capitalized and distinct from `False/0`, code that checks for it is self-documenting.

Disadvantages

- If you forget to check for `None`, later code that assumes a “real” value (like calling `.upper()` on a string) crashes with `AttributeError: 'NoneType' object has no attribute....` This is one of the most common runtime errors in Python.
- Silent `None` returns (a missing `else` branch, like above) are easy to miss during code review because nothing looks obviously wrong.

4. The Call Stack

Definition

The call stack is the mechanism Python uses to keep track of which function called which. Every time a function is called, a new “frame” holding its local variables and its return address is pushed onto the stack. When that function finishes, its frame is popped off, and control goes back to wherever it was called from. Because it’s a stack, it works last-in-first-out (LIFO): whichever function was called most recently is the first one to finish and return.

Example

```
def england():
    print('function a')
    australia()

def australia():
    print('function b')
    sa()

def sa():
    print('function e')
    ind()

def ind():
    print('function c')
    nz()

def nz():
    print('function d')

england()
```

Call order: england → australia → sa → ind → nz. Return order runs the other way: nz finishes first (nothing left to call), then ind, then sa, then australia, then england.

Output: function a, function b, function e, function c, function d

Why it works this way

Nested function calls need somewhere to “remember” where to resume once an inner call finishes. A stack is the natural structure for this because the most recently paused function is always the next one that needs to resume. That’s exactly LIFO behavior.

Advantages

- Lets functions call other functions (and themselves, recursion) without losing track of where each one needs to return to.
- Makes debugging traceable: a Python error traceback is literally a printout of the call stack at the moment the error happened, showing you the chain of calls that led there.

Disadvantages

- Deep or infinite recursion overflows the stack, raising `RecursionError: maximum recursion depth exceeded`.
- Long call chains can make execution flow hard to follow by reading code alone. You often need to actually trace or debug it to see the order.

5. Scope: Local and Global Variables

Definition

Scope determines where in your code a variable name is visible and usable. Python has (broadly) two scopes relevant here:

- **Local scope:** variables created inside a function. They exist only while that function is running and disappear afterward.
- **Global scope:** variables created outside any function, at the top level of a script or module. They exist for as long as the program runs and can be read from inside any function.

Example

```
def spam():
    egg = 'got from supermarket'    # local to spam()
    burger()
    print(egg)

def burger():
    beef = 'got from farm'          # local to burger()
    print(beef)

spam()
# beef is not visible inside spam(), and egg is not visible inside burger()
```

Reading a global from inside a function works without any special syntax:

```
def test_scope():
    print(egg)    # reads the global 'egg'

egg = 'global variable'
test_scope()
```

But *modifying* a global from inside a function requires the `global` keyword:

```
def practice_global_scope():
    global cow
    cow = 'biryani with beef'

cow = 'global biriyani'
practice_global_scope()
print(cow)    # 'biryani with beef', the global was actually changed
```

Without the `global` keyword, assigning to `cow` inside the function would instead create a brand-new *local* variable called `cow`, leaving the global one untouched.

Why Python enforces this

If every function could silently reach into and rewrite any variable anywhere in the program, tracking down where a value changed would become close to impossible in anything beyond a toy script.

Requiring `global` to be stated explicitly means that any time a function modifies shared state, that fact is visible right there in the function's own code. You don't have to read the entire program to know it's happening.

Advantages

- Local variables can reuse the same name in different functions (`beef` in `burger()`, `egg` in `spam()`) without any risk of collision, because they live in separate scopes.
- Requiring `global` for writes (not reads) makes accidental mutation of shared state harder, since you have to opt in.
- Keeping most variables local makes functions easier to test in isolation, since their behavior doesn't depend on hidden outside state.

Disadvantages

- Overusing global variables in medium-to-large programs makes bugs harder to trace, because any function could be the one that changed a given value.
- Forgetting the `global` keyword when you meant to modify a global variable is a classic, confusing bug: Python doesn't raise an error, it just quietly creates a new local variable instead, and your global never updates the way you expected.
- If you read a local variable before assigning it anywhere in the function (even later in the same function), Python raises `UnboundLocalError`, because Python decides a name is local for the *entire* function body if it's assigned anywhere inside it, even below the line that reads it.

6. Exception Handling (`try / except`)

Definition

Exceptions are Python's way of signaling that something went wrong while a program was running: dividing by zero, converting "abc" to an integer, opening a file that doesn't exist. A `try` block wraps code that might raise an exception. An `except` block tells Python what to do if a *specific* exception is raised, instead of letting the whole program crash.

Example

```
def count_number(number):
    try:
        return 50 / number
    except ZeroDivisionError:
        print("Error! Invalid argument, try any value except 0")

print(count_number(4))    # 12.5
print(count_number(0))    # prints the message, then returns None
```

A second example from the same idea, catching bad user input instead of bad math:

```
try:
    user_input = input('Enter any number')
    number = int(user_input)
except ValueError:
    print('you must enter integer number like 1,2,3')
```

Why catch a *specific* exception

`except ZeroDivisionError` only catches division-by-zero errors. A bare `except:` (with nothing after it) catches everything, including exceptions you didn't anticipate, like a typo causing a `NameError`, or the user hitting Ctrl+C. Catching everything blindly can hide real bugs by silently swallowing errors that should have stopped the program and told you something was wrong.

Advantages

- Programs keep running through predictable, recoverable errors (like bad input) instead of crashing outright.
- Naming the exact exception type documents exactly what failure mode you expected and planned for.
- Separates the “happy path” logic (the `try` block) from the error-handling logic (the `except` block), which keeps both easier to read.

Disadvantages

- Overusing broad exception handling can hide bugs: code that “works” only because errors are being silently caught and ignored.
- If the `except` block doesn't `return` anything (as in the example above), the function falls through to an implicit `None`, which can catch you off guard later if you forget that path exists.
- Exception handling adds a small amount of runtime cost compared to code with no error checking, though this rarely matters in practice.

7. Loops: `for`, `while`, and `break`

Definition

- A **for loop** iterates over a known sequence (a range of numbers, a list, a string) a fixed number of times.
- A **while loop** repeats as long as a condition stays true, including `while True:`, which repeats forever unless something inside explicitly stops it.
- **break** immediately exits the loop it's inside, skipping any remaining iterations.

Example: bounded `for` loop with `break`

```
import random
total_number = random.randint(1, 50)
```

```

for guess_taken in range(1, 11):      # 10 attempts
    guess = int(input('>'))
    if guess < total_number:
        print('too low')
    elif guess > total_number:
        print('too high')
    else:
        break      # correct guess, stop looping early

```

Example: infinite while loop, nested, with two different exits

```

import sys

while True:                          # outer: one game round per pass
    while True:                      # inner: keep asking until input is valid
        player_move = input('>')
        if player_move == 'q':
            sys.exit()              # ends the whole program
        if player_move in ('r', 'p', 's'):
            break                  # only exits the inner loop
        print('Type one of r, p, s, or q.')
    # ... play the round using player_move ...

```

Why two different loop types exist

for loops are for when you know, up front, what you’re iterating over: a fixed count, a list, a string’s characters. while loops are for when the stopping condition depends on something that happens *during* the loop: user input, a game being won, a network response arriving. Using `for` for the first case and `while` for the second makes the intent of the code visible just from which keyword was chosen.

Advantages

- `range(1, 11)` makes the loop bounded and safe by construction. It cannot run forever, unlike a `while` loop with a condition you might get wrong.
- `while True` with an internal `break` cleanly separates “how the loop normally continues” from “the one condition that stops it,” instead of cramming that logic into a complicated boolean condition at the top.
- Nesting loops lets you handle two separate concerns (like input validation vs. game logic) without tangling their code together.

Disadvantages

- `break` only exits the *innermost* loop it’s inside. Trying to exit an outer loop from deep inside nested loops with `break` alone doesn’t work. You need something else, like `sys.exit()`, a flag variable, or restructuring the code into a function you can `return` out of.
- `while True` loops are easy to get wrong: forgetting the `break` condition (or getting its logic backwards) creates a genuine infinite loop that hangs the program.

- Deeply nested loops (`while` inside `while` inside `if`) get harder to reason about the deeper they go. This is one of the more common sources of hard-to-spot logic bugs.
-

8. Handling Ctrl+C: KeyboardInterrupt

Definition

When a user presses `Ctrl+C` while a Python program is running, the operating system sends an interrupt signal, and Python turns that into a `KeyboardInterrupt` exception. By default, this exception is unhandled and stops the program immediately with a traceback. Wrapping the running code in `try / except KeyboardInterrupt` lets you intercept that moment and exit cleanly instead.

Example

```
import sys, time

try:
    while True:
        print('*****')
        time.sleep(0.1)
except KeyboardInterrupt:
    sys.exit()
```

Why this matters for infinite loops

Animation-style programs (progress bars, ASCII art, anything meant to run until the user says stop) are usually built as intentional `while True:` loops with no built-in stopping condition. `KeyboardInterrupt` is the standard way a user tells such a program to stop. Catching it explicitly means you control what happens at shutdown: printing a goodbye message, cleaning up a file handle, or, as above, just exiting quietly instead of dumping a traceback to the screen.

Advantages

- Gives infinite loops a clean, expected way to end, controlled by the user rather than by a bug.
- Prevents a scary-looking traceback from printing every time someone stops a perfectly normal, working program.

Disadvantages

- If overused to catch `KeyboardInterrupt` in places where the user is trying to genuinely abort a stuck or broken program, it can make the program feel unresponsive or hard to kill.
 - Any cleanup code has to be written correctly inside the `except` block. If it isn't, a program can leave things in a half-finished state even though it appeared to “exit cleanly.”
-

9. `sys.exit()` vs. `break` vs. `return`

Definition

All three stop something from continuing, but at different scopes:

- **return** exits the *current function only*, optionally handing back a value.
- **break** exits the *current loop only* (the innermost one it's inside), and nothing more.
- **sys.exit()** ends the *entire program*, immediately, no matter how deeply nested the code currently is.

Example

```
import sys

while True:
    move = input('>')
    if move == 'q':
        sys.exit()      # program ends here, full stop
    if move in ('r', 'p', 's'):
        break           # only exits this while loop
```

Why three separate tools exist

Each solves a different problem. **return** is about a function's own control flow. **break** is about a loop's control flow. Neither one is built to reach further out than its own construct. A **break** inside a loop that's inside a function cannot, by itself, also make that function return. **sys.exit()** exists specifically for the case where you need to override all of that and stop everything, regardless of how many functions or loops are currently active.

Advantages

- Having distinct tools for distinct scopes keeps control flow predictable: **break** will never accidentally end the whole program, and **sys.exit()** will never accidentally just continue the loop.
- **sys.exit()** is the simplest way to implement a “quit” command from deep inside nested logic, without threading a “should I stop?” flag through every layer.

Disadvantages

- **sys.exit()** is a blunt tool. It skips over any code that would otherwise run after the loop or function, which can bypass cleanup you actually wanted to happen (unless handled with **try/finally**).
- Beginners often reach for **sys.exit()** when a plain **return** or **break** was what they actually needed, which can make small scripts harder to reason about than necessary.

10. The random Module

Definition

`random` is a built-in Python module for generating pseudo-random values: random integers, random choices from a list, shuffling, and more. It's “pseudo” random because it's generated by a deterministic algorithm seeded from something like the system clock, not truly random, but unpredictable enough for games, simulations, and sampling.

Example

```
import random

total_number = random.randint(1, 50)    # random int, 1 and 50 both possible
move_number = random.randint(1, 3)      # random int from 1, 2, or 3
```

Why `randint` is inclusive on both ends

`random.randint(a, b)` was designed to match how people naturally describe a range in conversation. “A number between 1 and 50” usually means both 1 and 50 are valid answers. This is different from Python's `range(a, b)`, which excludes `b`. The inconsistency between the two is a common source of off-by-one mistakes, so it's worth memorizing explicitly rather than guessing.

Advantages

- Makes games, simulations, and sampling straightforward without implementing your own randomness algorithm.
- `randint`, `choice`, `shuffle`, and similar functions cover most common use cases with a single, readable line of code.

Disadvantages

- Not cryptographically secure. `random` should never be used to generate passwords, tokens, or anything security-sensitive. Python has a separate `secrets` module for that.
- Because it's pseudo-random and seedable, results can be reproduced exactly if you know the seed, which is useful for testing but easy to forget when you actually need true unpredictability.

11. String Repetition and Formatting

Definition

Python lets you “multiply” a string by an integer to repeat it: `'-' * 3` produces `---`. Combined with a loop variable, this is a quick way to build growing or shrinking visual patterns without manually concatenating strings.

Older-style string formatting uses the `%` operator: `'%s Wins, %s Losses' % (wins, losses)` substitutes each `%s` with the corresponding value from the tuple, in order.

Example

```
for i in range(1, 9):
    print('-', (i * i))    # 1, 4, 9, 16, 25, 36, 49, 64 dashes
```

Why this works

`*` is overloaded in Python: for two numbers it multiplies, but for a string and an integer, it repeats the string that many times. This is possible because Python lets types define their own behavior for operators (through methods like `__mul__`), and `str` chose to define multiplication as repetition since it's a natural, common operation on sequences.

Advantages

- Extremely concise for building patterns, progress bars, separators, and simple visual output.
- Readable at a glance once you know the idiom: `'-' * n` clearly means “n dashes” without a loop.

Disadvantages

- %-style formatting is considered legacy; modern Python code generally prefers f-strings (`f'{wins} Wins, {losses} Losses'`), which are more readable and let you embed expressions directly.
- String repetition with a large multiplier (accidentally, from a bug) can produce enormous strings and eat memory fast, with no warning.

12. Mutable vs. Immutable

Definition

Mutable objects can be changed in place after they're created. Their identity (the object in memory) stays the same while their contents change. **Immutable** objects cannot be changed once created. Any operation that looks like it's “modifying” one actually builds a brand-new object and points the variable at that instead.

Common immutable types: `int`, `float`, `str`, `tuple`, `bool`, `frozenset`. Common mutable types: `list`, `dict`, `set`.

Example

```
# Immutable: strings can't be changed in place
name = 'fahim'
name += '6a'    # this creates a NEW string, 'name' now points to it
                # the original 'fahim' string still exists unchanged
                # elsewhere in memory until nothing references it

# Mutable: lists CAN be changed in place
```

```
scores = [10, 20, 30]
scores.append(40)      # the same list object is modified
print(scores)          # [10, 20, 30, 40]
```

The difference becomes visible with `id()`:

```
x = 'hello'
print(id(x))
x += ' world'
print(id(x))           # different id: a new string object was created

y = [1, 2]
print(id(y))
y.append(3)
print(id(y))           # same id: the original list object was changed
```

Why this distinction exists

Immutability gives Python guarantees it can rely on. A string or a tuple can be safely used as a dictionary key or stored in a set, because its value (and therefore its hash) can never change out from under you. Mutability, on the other hand, is what makes lists and dictionaries useful as growing, shrinking containers you build up over time, without recreating the whole thing on every change.

Advantages

- **Immutable:** safe to share between functions without one function accidentally altering data another part of the program depends on. Also safe to use as dictionary keys and set members.
- **Mutable:** efficient for building up or modifying large collections, since you don't need to create a full copy every time you add or remove something.

Disadvantages

- **Immutable:** any “modification” (like `name += '6a'` above) creates a new object, which can be wasteful if done repeatedly in a loop, such as building a long string one character at a time.
- **Mutable:** passing a mutable object into a function and modifying it there changes the original object everywhere else it's referenced too. This is a common source of unintended side effects, especially with default mutable arguments (see Section 18).

13. Lists vs. Tuples

Definition

Both are ordered collections that can hold items of any type, accessed by index. The core difference: a **list** (`[]`) is mutable, a **tuple** (`()`) is immutable.

Example

```
fruits_list = ['apple', 'banana', 'cherry']
fruits_list[0] = 'mango'           # works fine, lists are mutable
fruits_list.append('grape')        # works fine

fruits_tuple = ('apple', 'banana', 'cherry')
fruits_tuple[0] = 'mango'          # TypeError: 'tuple' object does not support item assignment
```

Tuples are commonly used to group fixed, related values together:

```
point = (3, 4)                    # x, y: this pairing is never meant to change
x, y = point                       # unpacking
```

Why both exist

A list says “this is a changeable collection I’ll be adding to, removing from, or rearranging.” A tuple says “this is a fixed group of values, treat it as a single unit.” Choosing one over the other communicates intent to anyone reading the code, and it isn’t just a style choice: because tuples are immutable, they can be hashed, which means they (unlike lists) can be used as dictionary keys or stored inside a set.

Advantages

- **List:** flexible, supports adding, removing, sorting, and reversing in place.
- **Tuple:** faster to create and iterate over than a list of the same size, since Python doesn’t need to allocate extra room for future growth. Safe to use as a dictionary key.

Disadvantages

- **List:** cannot be used as a dictionary key or set member, because it’s mutable (and therefore unhashable).
 - **Tuple:** once created, you’re stuck with its contents. Even changing a single value means building an entirely new tuple.
-

14. Dictionaries

Definition

A dictionary (`dict`) stores data as key-value pairs. Instead of looking items up by numeric position like a list, you look them up by a key, and each key must be unique within the dictionary. Since Python 3.7, dictionaries also preserve the order in which keys were inserted.

Example

```
player = {
    'name': 'Ben Stokes',
```

```

    'role': 'all-rounder',
    'team': 'England'
}

print(player['name'])          # 'Ben Stokes'
player['team'] = 'England Test' # update an existing key
player['debut_year'] = 2011     # add a new key

for key, value in player.items():
    print(key, ': ', value)

# .get() avoids a crash if the key doesn't exist
print(player.get('age', 'unknown')) # 'unknown', no KeyError raised

```

Why dictionaries exist

Looking something up by a meaningful name (`player['name']`) is both faster and more readable than looking it up by position (`player[0]`) once your data has more than a couple of fields. Under the hood, a dictionary uses a hash table, so looking up a key takes roughly the same amount of time whether the dictionary has 10 entries or 10 million.

Advantages

- Lookup, insertion, and deletion by key are all, on average, constant-time operations ($O(1)$), regardless of how large the dictionary grows.
- Self-documenting: `player['name']` is clearer than remembering that index 0 of some list means “name.”
- Keys can be any immutable type (strings, numbers, tuples), not just strings.

Disadvantages

- Every key must be unique. Assigning to an existing key overwrites the old value instead of adding a second one.
- Only immutable, hashable types can be used as keys. Trying to use a list as a key raises `TypeError: unhashable type: 'list'`.
- Slightly more memory overhead per item than a list, because of the hash table structure underneath.

15. Sets

Definition

A set is an unordered collection of unique items. Adding a duplicate value to a set has no effect, since sets automatically discard duplicates. Sets are built on the same hash-table idea as dictionaries (in fact, a set is close to being “a dictionary with only keys, no values”).

Example

```
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)           # {1, 2, 3}, duplicates removed automatically

a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)              # {2, 3}, intersection
print(a | b)              # {1, 2, 3, 4}, union
print(a - b)              # {1}, items in a but not in b
```

Why sets exist

Two problems come up constantly: “does this collection contain duplicates I need to remove?” and “is this specific item present at all?” A list can answer both, but checking membership in a list (`x in my_list`) has to check every item one by one in the worst case. A set answers both in roughly constant time, because membership checking uses the same hashing mechanism as a dictionary lookup.

Advantages

- Removing duplicates from a collection is as simple as `set(my_list)`.
- Membership testing (`x in my_set`) is much faster than in a list for large collections.
- Built-in support for mathematical set operations: union, intersection, difference.

Disadvantages

- Sets are unordered, so you can’t rely on any particular order when iterating, and you can’t access items by index.
 - Only hashable (generally immutable) items can go inside a set, so you can’t put a list inside a set directly.
-

16. Strings, Slicing, and f-strings

Definition

A string is an immutable sequence of characters. Because it’s a sequence, it supports indexing and **slicing** (pulling out a sub-section using `start:stop:step` notation). **f-strings** (formatted string literals, `f'...'`) are the modern way to embed variables and expressions directly inside a string.

Example

```
name = 'Fahim'
print(name[0])           # 'F'
print(name[-1])          # 'm', negative index counts from the end
print(name[1:4])         # 'ahi', slice from index 1 up to (not including) 4
print(name[::-1])        # 'mihaF', reversing a string with a step of -1
```

```
age = 25
print(f'{name} is {age} years old')      # f-string
print('%s is %s years old' % (name, age)) # old-style, still valid but discouraged
```

Why slicing works this way

`start:stop:step` was designed so `stop - start` (when `step` is 1) tells you exactly how many characters you get, which keeps slice-length arithmetic simple and consistent across strings, lists, and tuples alike. The “stop is exclusive” rule matches how `range()` behaves for the same reason.

Advantages

- Slicing is a concise way to grab substrings, reverse a sequence, or skip every other character, without writing a loop.
- f-strings are readable and let you embed any expression, not just a variable name: `f'{price * 1.1:.2f}'`.

Disadvantages

- Off-by-one confusion is common with slicing until the “stop is exclusive” rule becomes second nature.
- Because strings are immutable, operations like repeatedly slicing and rebuilding a string in a loop can be slow for very large strings; joining a list of pieces (`''.join(pieces)`) is faster.

17. Comprehensions

Definition

A comprehension builds a new list, dictionary, or set in a single, compact line by looping over an existing iterable and optionally filtering it, instead of writing a full `for` loop with `.append()` calls.

Example

```
# List comprehension
squares = [n * n for n in range(1, 6)]      # [1, 4, 9, 16, 25]

# With a filter condition
evens = [n for n in range(10) if n % 2 == 0] # [0, 2, 4, 6, 8]

# The equivalent written as a normal loop
evens = []
for n in range(10):
    if n % 2 == 0:
        evens.append(n)

# Dict comprehension
```

```
squares_dict = {n: n * n for n in range(1, 4)}    # {1: 1, 2: 4, 3: 9}

# Set comprehension
unique_lengths = {len(word) for word in ['hi', 'python', 'ok']}    # {2, 6}
```

Why comprehensions exist

The “loop, optionally filter, build a new collection” pattern is extremely common, and writing it out as a full `for` loop with `.append()` every time is repetitive. A comprehension reads, once you’re used to the syntax, almost like an English description of the result: “squares of `n`, for each `n` from 1 to 5.”

Advantages

- Shorter and, once familiar, often more readable than the equivalent loop.
- Generally a little faster than the loop-and-append version, since the looping is handled internally by Python rather than through repeated method calls.

Disadvantages

- Nesting multiple loops or conditions inside a single comprehension quickly becomes hard to read, undoing the readability benefit.
- Beginners often reach for a comprehension when a plain loop would be clearer, especially once the logic inside gets more complex than a single expression.

18. *args, **kwargs, and Default Arguments

Definition

- A **default argument** gives a parameter a fallback value, used only if the caller doesn’t supply one.
- ***args** collects any number of extra positional arguments into a tuple.
- ****kwargs** collects any number of extra keyword arguments into a dictionary.

Example

```
def greet(name, greeting='Hello'):           # default argument
    print(f'{greeting}, {name}!')

greet('Fahim')                               # uses default: 'Hello, Fahim!'
greet('Fahim', 'Hi')                         # overrides default: 'Hi, Fahim!'

def total(*args):                             # collects (2, 4, 6) into a tuple
    return sum(args)

print(total(2, 4, 6))                        # 12
```

```
def describe(**kwargs):          # collects into a dict
    for key, value in kwargs.items():
        print(key, '=', value)

describe(name='Fahim', role='developer')
```

The mutable-default-argument trap

```
def add_item(item, basket=[]):    # DANGER: default list is created ONCE
    basket.append(item)
    return basket

print(add_item('apple'))         # ['apple']
print(add_item('banana'))       # ['apple', 'banana'], not a fresh list!
```

Default argument values are evaluated **once**, when the function is defined, not each time it's called. A mutable default (like `[]`) is therefore shared across every call that doesn't pass its own value, silently accumulating state between unrelated calls. The standard fix is to default to `None` and create the list inside the function:

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket
```

Why these exist

Real functions often need to accept a variable, unpredictable number of inputs, think `print()`, which can take any number of arguments. `*args` and `**kwargs` let a function's signature stay flexible without you having to guess and pre-declare a fixed number of parameters.

Advantages

- Default arguments let common cases stay simple (`greet('Fahim')`) while still allowing customization when needed.
- `*args/**kwargs` let library and framework code (decorators, wrappers) forward arguments they don't need to know about in advance.

Disadvantages

- Mutable default arguments are one of the most common real-world Python bugs, precisely because the behavior is unintuitive until you've been bitten by it once.
- Overusing `*args/**kwargs` in your own functions can hide what a function actually expects, making it harder for callers (and IDEs) to know what to pass.

19. `is` vs. `==`, and Shallow vs. Deep Copy

Definition

`==` checks whether two objects have equal *values*. `is` checks whether two names point to the exact same object in memory (identity). Related: a **shallow copy** duplicates a container but not the objects inside it; a **deep copy** duplicates the container and everything inside it, recursively.

Example

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)      # True, same values
print(a is b)      # False, two different list objects in memory

c = a
print(a is c)      # True, 'c' is just another name for the same object

import copy
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)

original[0][0] = 99
print(shallow[0][0]) # 99, shallow copy shares the inner lists
print(deep[0][0])    # 1, deep copy is fully independent
```

Why the distinction matters

`==` answers “do these look the same?” `is` answers “are these literally the same thing?” Using `is` to compare values (if `x is 5:`) is a common beginner mistake, since it can behave inconsistently: small integers and short strings are sometimes cached and reused by Python internally, so `is` might accidentally “work” for them and then fail for larger numbers or different objects. The one place `is` is the *correct* tool is comparing against `None` (if `x is None:`), because there is only ever one `None` object in the entire program.

Advantages

- `==` correctly compares values regardless of whether they’re the “same” object, which is what you almost always want.
- `is None` is a fast, unambiguous check that can never be fooled by an object that happens to be falsy but isn’t actually `None` (like 0 or an empty list).
- Shallow copies are cheap and sufficient when a container’s contents are themselves immutable or don’t need independence.

Disadvantages

- Using `is` for value comparison instead of identity comparison is a classic, hard-to-spot bug.

- Shallow copies can cause the same kind of accidental-shared-state bug as mutable default arguments: changing a nested object inside the “copy” also changes the original.
- Deep copies are more expensive in time and memory, since every nested object has to be recursively duplicated.

20. A Few More Interview Staples

A handful of smaller, frequently-asked topics that don’t each need a full section:

class and self. A class is a blueprint for creating objects that bundle data (attributes) and behavior (methods) together. **self** is the conventional name for the first parameter of any method, referring to the specific object the method was called on. `__init__` is the method Python runs automatically right after a new object is created, typically used to set up its initial attributes.

```
class Player:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def introduce(self):
        return f'{self.name} plays as {self.role}'

stokes = Player('Ben Stokes', 'all-rounder')
print(stokes.introduce())
```

Iterators and generators. An iterable is anything you can loop over (`list`, `str`, `dict`). An iterator is the object that actually produces items one at a time, remembering its position between calls. A generator is a simple way to write an iterator using a function with `yield` instead of `return`, producing values lazily, one at a time, instead of building the whole sequence in memory up front.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for number in countdown(3):
    print(number)    # 3, 2, 1, computed one at a time, not stored as a full list
```

map, filter, and lambda. A `lambda` is a small, unnamed function defined in a single expression, useful when a full `def` would be overkill. `map()` applies a function to every item in an iterable; `filter()` keeps only the items where a function returns `True`.

```
nums = [1, 2, 3, 4, 5]
doubled = list(map(lambda x: x * 2, nums))    # [2, 4, 6, 8, 10]
evens = list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
```

Common built-ins worth knowing cold: `len()`, `range()`, `enumerate()` (loop with both index and value), `zip()` (pair up two sequences element-by-element), `sorted()`, `type()`, `isinstance()`.

```
names = ['Fahim', 'Stokes', 'Root']
for index, name in enumerate(names):
    print(index, name)

scores = [90, 85, 77]
for name, score in zip(names, scores):
    print(name, score)
```

Why these matter in interviews: they show up constantly in “write a function that...” questions, and interviewers often use them as a quick check for whether you actually write Python day-to-day, versus only knowing the syntax you memorized for a test.

Quick-reference table

Concept	One-line definition	Common pitfall
Function	Reusable named block of code	Relying on global state instead of parameters
Parameter vs. argument	Placeholder name vs. actual value passed	Using the terms interchangeably in an interview
Return value / None	Value handed back; None if nothing is returned	Missing else /branch silently returns None
Call stack	LIFO record of active function calls	Infinite recursion → RecursionError
Local scope	Variables that exist only inside a function	Assuming a local variable is visible elsewhere
Global scope	Variables visible everywhere, read by default	Forgetting global when trying to modify one
try / except	Catch a specific error instead of crashing	Bare except : hides real bugs
for loop	Fixed number of iterations over a known sequence	Off-by-one errors with range()
while True	Repeats until something inside stops it	Forgetting the break condition = infinite loop
break	Exits the innermost loop only	Expecting it to exit an outer loop too
KeyboardInterrupt	Exception raised by Ctrl+C	Catching it in places where users need to force-quit
sys.exit()	Ends the entire program immediately	Skips cleanup code unless using try/finally
random.randint(a, b)	Random integer, inclusive on both ends	Confusing it with range() , which excludes the end
str * int	Repeats a string that many times	Runaway memory use from a large accidental multiplier
Mutable	Can be changed in place after creation	Shared references change unexpectedly everywhere

Concept	One-line definition	Common pitfall
Immutable	Cannot be changed once created	“Modifying” actually creates a new object each time
List	Ordered, mutable collection	Can’t be used as a dict key or set member
Tuple	Ordered, immutable collection	Can’t change a single value without rebuilding it
Dictionary	Key-value pairs, $O(1)$ average lookup	Keys must be unique and hashable (immutable)
Set	Unordered collection of unique items	No indexing, no guaranteed order
Slicing	<code>seq[start:stop:step]</code>	<code>stop</code> is exclusive, off-by-one errors are common
f-string	<code>f'{variable}'</code> embedded formatting	Only in Python 3.6+
Comprehension	One-line loop + filter to build a collection	Deep nesting kills readability
<code>*args</code> / <code>**kwargs</code>	Collect extra positional/keyword arguments	Overuse hides what a function actually expects
Mutable default argument	<code>def f(x=[]):</code>	Default is created once, shared across all calls
<code>==</code> vs <code>is</code>	Value equality vs. identity	Using <code>is</code> for values instead of <code>is None</code>
Shallow vs. deep copy	Copies the container vs. copies everything inside too	Shallow copy still shares nested mutable objects
<code>class</code> / <code>self</code> / <code>__init__</code>	Blueprint for objects; the object itself; setup method	Forgetting <code>self</code> as the first method parameter
Generator (<code>yield</code>)	Produces values one at a time, lazily	Can only be iterated over once
<code>map()</code> / <code>filter()</code> / <code>lambda</code>	Apply/filter a function across an iterable; anonymous function	Overuse instead of a plain, more readable loop